PRAKSE · P25-2 · 3. DIENA

Serveris, kas glabā rekordus

Mērķis šodien: serveris, kuram var iedot rekordu, un tas to atceras arī pēc restarta. Spēli ar to savienosim rīt.

KAS IR SERVERIS

Trīs daļas, kas strādā kopā kā restorāns

KLIENTS · RĪT

Spēle

Spēlē un iegūst punktus. Sūta tos serverim.



VIESMĪLIS · ŠODIEN

Serveris (API)

Pieņem punktus, pārbauda tos, atdod top-10.



NOLIKTAVA · ŠODIEN

Datubāze

Glabā visus rekordus arī pēc restarta.

Kāpēc vajag viesmīli? Ja spēle rakstītu tieši datubāzē, jebkurš varētu sev ierakstīt miljonu punktu.

KĀ RUNĀ AR API

Pieprasījums un atbilde

PIEPRASĪJUMS (KLIENTS → SERVERIS)

`POST /scores`

Dati JSON: {"player": "Anna", "score": 900}

ATBILDE (SERVERIS → KLIENTS)

`201`

Saglabāts. Un atpakaļ tie paši dati.

Kods	Nozīme	Kad
200	Kārtībā	Dati nolasīti
201	Izveidots	Rekords saglabāts
404	Nav atrasts	Tāda spēlētāja nav
422	Slikti dati	Tukšs vārds, −5 punkti

GET = „iedod man”, POST = „še, saglabā”

KUR KAS JĀRAKSTA

Mape api/: adreses vienā failā, datubāze otrā

`api/main.py`

Adreses (endpointi): ko serveris saprot un ko atbild.

`api/db.py`

Savienojums ar datubāzi un tabulas izveide.

`game.db`

Pats datubāzes fails. Nenonāk GitHub: .gitignore → *.db

Palaiž no projekta mapes:

```
uvicorn api.main:app --reload
```

Pārbauda pārlūkā:

```
localhost:8000/docs
```

/docs ir tava pārbaudes vieta: tur katru adresi var izmēģināt ar pogu, bez spēles.

1. SOLIS NO 4

Serveris palaižas, un /docs atveras

PĒC ŠĪ SOĻA TAVS SERVERIS PROT

- FastAPI un uvicorn ir uzstādīti un ierakstīti requirements.txt
- Mapē api/ ir main.py ar vienu adresi, piemēram /hello
- Serveris palaižas un neizmet kļūdu
- localhost:8000/docs lapā redzama tava adrese

Ko meklēt dokumentācijā

`FastAPI()` `@app.get` `uvicorn` `--reload` `/docs`

PADOMS

Adrese ir parasta Python funkcija ar „uzlīmi" virs tās, kas pasaka metodi un adresi. Ko funkcija atgriež, to saņem klients.

KĀ PĀRBAUDĪT

Atver /docs, izvēlies savu adresi → Try it out → Execute. Redzi atbildi un kodu 200?

Rekordu var pievienot un top-10 nolasīt

PĒC ŠĪ SOĻA TAVS SERVERIS PROT

- POST /scores pieņem vārdu un punktus un atbild ar 201
- GET /scores/top atdod 10 labākos, lielākie punkti pirmie
- Pagaidām rekordi glabājas parastā Python sarakstā

Ko meklēt dokumentācijā

`@app.post` `status_code=201` `pydantic BaseModel` `sorted` `reverse=True`

PADOMS

To, ko klients sūta, apraksta ar modeli: vārds ir teksts, punkti ir skaitlis. FastAPI pats pārbauda tipus un uz nepareiziem datiem atbild ar 422.

KĀ PĀRBAUDĪT

/docs: pievieno 3 rekordus ar dažādiem punktiem. Top rāda pareizā secībā? Restartē serveri: rekordi pazuda? Šajā solī tā tam jābūt.

Rekordi glabājas datubāzē

PĒC ŠĪ SOĻA TAVS SERVERIS PROT

- Fails api/db.py savienojas ar SQLite un izveido tabulu
- POST /scores ieraksta rekordu datubāzē
- GET /scores/top nolasa no datubāzes, sakārtotu
- Datubāzes fails nenonāk GitHub

Ko meklēt dokumentācijā

`sqlite3.connect` `CREATE TABLE IF NOT EXISTS` `INSERT INTO` `ORDER BY ... DESC` `LIMIT` `?`

PADOMS

Nekad neliec vērtības SQL tekstā ar f-string. Lieto ? vietas. Citādi kāds var uzlauzt datubāzi (SQL injekcija).

KĀ PĀRBAUDĪT

Pievieno rekordus, aptur serveri (Ctrl+C), palaid vēlreiz. Rekordi joprojām ir? Un git status nerāda .db failu?

4. SOLIS NO 4

Sliktie dati tiek noraidīti, statistika strādā

PĒC ŠĪ SOĻA TAVS SERVERIS PROT

- Tukšs vārds, pārāk garš vārds vai negatīvi punkti → 422, nekas netiek ierakstīts
- GET /players/{name}/stats atdod spēļu skaitu un labāko rezultātu
- Ja tāda spēlētāja nav → 404

Ko meklēt dokumentācijā

`pydantic Field` `min_length` `max_length` `ge=0`

`HTTPException` `COUNT` `MAX`

PADOMS

Robežas var ierakstīt pašā modelī: minimālais un maksimālais garums, punkti ne mazāki par 0. Nav jāraksta if katram gadījumam.

KĀ PĀRBAUDĪT

/docs: sūti −5 punktus un tukšu vārdu, abiem 422? Pieprasi statistiku vārdam, kura nav: 404? Un esošam: pareizi skaitļi?

KAD IESTRĒGSTI

Kur meklēt palīdzību, pirms pacel roku

1

Skaties terminālī

Īstā servera kļūda ir tur, kur palaists uvicorn.

2

Dokumentācija

fastapi.tiangolo.com → Tutorial

3

Izmēģini /docs

Katru adresi var pārbaudīt ar pogu.

4

Paceļ roku

Ko redzi	Ko darīt
No module named 'api'	Palaid uvicorn no projekta mapes; mapē api/ jābūt __init__.py
address already in use	Serveris jau darbojas citā terminālī: Ctrl+C tur
422 visu laiku	JSON lauku nosaukumi nesakrīt ar modeļa laukiem
500 Internal Server Error	Lasi kļūdu terminālī, kur palaists uvicorn

DIENAS NOSLĒGUMS

Viena minūte katram

1

Parādi /docs

Pievieno rekordu → top → statistika

2

Ko darīšu rīt

Vienā teikumā

3

Kur iestrēgu

Vai „nekur”

Pirms aiziešanas: commit un push. Rīt: spēle sūta rekordus serverim, un tie parādās mājaslapā.